BÁO CÁO THỰC HÀNH

Thực hành an toàn Mạng máy tính

Lab 02: Modern Cryptography

GVTH: KS.Nguyễn Ngọc Trưởng

Ngày báo cáo: 23/03/2026

Nhóm 08 – NT101.Q21.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24520115	Nguyễn Quang Thế Anh	Tất cả	100%

1. Cấu trúc Feistel và Sự lan truyền thay đổi

1. Ý tưởng thực hiện:
   a. Định lý: Cấu trúc mật mã Feistel
   Feistel đề xuất rằng chúng ta có thể xấp xỉ một mật mã khối lý tưởng bằng cách sử dụng khái niệm mật mã tích (product cipher), tức là luân phiên các phép thế (substitutions) và hoán vị (permutations). Điều này có nghĩa là thực hiện hai hoặc nhiều hệ mật mã đơn giản một cách tuần tự sao cho kết quả cuối cùng có độ mạnh mật mã cao hơn bất kỳ thành phần nào. Cấu trúc mật mã Feistel được sử dụng bởi DES và một số hệ mật mã khối đối xứng quan trọng hiện đang được sử dụng (ví dụ: TDEA, TLS).
   b. Nhiệm vụ:
   Dựa trên Định lý về cấu trúc mật mã Feistel, hoàn thiện hàm feistel_round. Sau đó, thực hiện mã hóa hai bản rõ M1 = 0xAB và M2 = 0xAC (chỉ khác nhau 1-bit cuối).
   c. Yêu cầu:
   In giá trị (L, R) sau mỗi vòng của cả hai quá trình và nhận xét về sự khác biệt giữa chúng tăng dần qua từng vòng như thế nào.
   d. Gợi ý:
   Công thức mỗi vòng: $LE_i = RE_{i-1}$ và $RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$. Với F là hàm nội bộ kết hợp XOR và dịch bit [2].
2. Để giải quyết yêu cầu của bài toán:
- Xây dựng hàm giả ngẫu nhiên (Pseudorandom Function) với tên hàm là F được tham số hóa bởi khóa K_i.
- Xây dựng hàm thực thi 1 vòng của cấu trúc feistel với tên hàm là feistel_round.
- Xây dựng hàm để theo dõi sự lan truyền thay đổi qua nhiều vòng lặp với tên hàm là track_avalanche.
3. Ngôn ngữ sử dụng: Python 3.14
4. Các bước thực hiện:

Bước 1: Xây dựng hàm giả ngẫu nhiên.

Ý tưởng: có dữ liệu đầu vào là nữa bên phải của dữ liệu (right) và khóa con (subkey) và trả ra kết quả dữ liệu ở dạng nhị phân có 4-bit.

- Thực hiện phép tính XOR giữa right và subkey làm cho dữ liệu phụ thuộc vào khóa nếu 1-bit khóa thay đổi thì đầy ra cũng thay đổi.
- Sau đó tiếp tục thực hiện phép toán AND giữa kết quả của Phép toán XOR với 0x0F (0x0F là mã ở dạng thập lục phân là số 15 trong hệ thập phân và có độ dài 4-bit trong hệ nhị phân 1111) với mục đích luôn giữ cho kết quả luôn trong phạm vi 4-bit.
- Trả về kết quả là dãy nhị phân có độ dài 4-bit.

```python
def F(right, subkey):
    return (right ^ subkey) & 0x0F
```

Hình 1.1: Phần code bên trong hàm F.

Bước 2: Xây dựng hàm thực thi một vòng của cấu trúc feistel.

Ý tưởng: đầu vào bao gồm nữa dữ liệu bên phải (R_in), nữa dữ liệu bên trái (L_in) và khóa con (subkey) và trả ra kết quả nửa dữ liệu mới bên trái và phải (L_out, R_out) sau khi thực hiện 1 số phép toán trên dữ liệu cũ.

- Cập nhật giá trị nửa bên trái của vòng tiếp theo bằng nửa bên phải của vòng hiện tại ($LE_i = RE_{i-1}$).
- Cập nhật giá trị nửa bên phải của vòng tiếp theo bằng cách lấy nửa bên trái hiện tại bằng phép toán XOR với kết quả của hàm F ($RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$).
- Trả về giá trị của 2 biến nửa bên trái và phải (L_out, R_out).

```python
def feistel_round(L_in, R_in, subkey):
    L_out = R_in
    R_out = L_in ^ F(R_in, subkey)
    return L_out, R_out
```

Hình 1.2: Phần code logic bên trong hàm thực thi một vòng của cấu trúc feistel.

Bước 3: Xây dựng hàm để theo dõi sự lan truyền thay đổi qua nhiều vòng lặp.

Ý tưởng: dữ liệu đầu vào là dạng nhị phân có độ dài 8-bit và khóa key.

- Biến L ở dạng nhị phân có độ dài 4-bit bên trái của giá trị đầu vào và biến R cũng như biến L chỉ khác là có giá trị là chuỗi 4-bit bên phải của giá đầu vào.
- 'msg >> 4' có nghĩa là đầu vào được dịch phải 4-bit sau đó sử dụng phép toán AND với 0x0F để có giá trị là 4-bit.
- Ví dụ msg = 0xAB = 10101011, L = 1010, R = 1011.
- Tiếp theo tạo khóa con (subkey), sinh ra 4 khóa cho 4 vòng lặp.
- Tạo ra vòng lặp chạy qua 4 vòng feistel in ra kết quả ở từng vòng lặp, cập nhật giá trị mới cho 2 biến L và R để xem sự thay đổi của bit.
- Gộp nửa trái và nửa phải lại thành kết quả mã hóa cuối cùng bằng phép dịch bit trái (<<) và phép toán OR (|).
- Sau khi kết thúc hàm track_avanlanche để theo dõi sự lan truyền thay đổi qua nhiều vòng lặp thì thực hiện in ra kết quả với các ví dụ để xem kết quả

```python
def track_avalanche(msg, key):
    L, R = (msg >> 4) & 0x0F, msg & 0x0F
    subkeys = [
        key & 0x0F,
        (key >> 4) & 0x0F,
        (key + 1) & 0x0F,
        (key + 2) & 0x0F
    ]

    print(f"Khởi tạo: L={format(L, '04b')}, R={format(R, '04b')}")
    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f"Vòng {i+1}: L={format(L, '04b')}, R={format(R, '04b')}")
    return (L << 4) | R
```

Hình 1.3: Phần code logic bên trong hàm để theo dõi sự lan truyền thay đổi qua nhiều vòng lặp.

```python
def F(right, subkey):
    return (right ^ subkey) & 0x0F

def feistel_round(L_in, R_in, subkey):
    L_out = R_in
    R_out = L_in ^ F(R_in, subkey)
    return L_out, R_out

def track_avalanche(msg, key):
    L, R = (msg >> 4) & 0x0F, msg & 0x0F
    subkeys = [
        key & 0x0F,
        (key >> 4) & 0x0F,
        (key + 1) & 0x0F,
        (key + 2) & 0x0F
    ]

    print(f"Khởi tạo: L={format(L, '04b')}, R={format(R, '04b')}")
    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f"Vòng {i+1}: L={format(L, '04b')}, R={format(R, '04b')}")
    return (L << 4) | R

print("--- Mã hóa M1 (0xAB) ---")
track_avalanche(0xAB, 0x12)
print("\n--- Mã hóa M2 (0xAC) ---")
track_avalanche(0xAC, 0x12)
```

Hình 1.4: Phần code hoàn chỉnh của cấu trúc feistel và sự lan truyền thay đổi.

Bước 4: Thực hiện chạy code và in ra kết quả.

```
PS D:\antoanmangmaytinh> & C
sk2_1.py
--- Mã hóa M1 (0xAB) ---
Khởi tạo: L=1010, R=1011
Vòng 1: L=1011, R=0011
Vòng 2: L=0011, R=1001
Vòng 3: L=1001, R=1001
Vòng 4: L=1001, R=0100

--- Mã hóa M2 (0xAC) ---
Khởi tạo: L=1010, R=1100
Vòng 1: L=1100, R=0100
Vòng 2: L=0100, R=1001
Vòng 3: L=1001, R=1110
Vòng 4: L=1110, R=0011
PS D:\antoanmangmaytinh>
```

Hình 1.5: Kết quả sau khi chạy code trong file python.

Giải thích:

- Đầu vào:
  - Bản rõ M1 = 0xAB và key là 0x12
  - M1 = 0xAB = 10101011
  - Bản rõ M2 = 0xAC và key là 0x12
  - M2 = 0xAC = 10101100
- Đầu ra của bản rõ M1 và key với bản rõ M2 thì tương tự M1:
  - Khởi tao: L=1010, R=1011 do L chỉ lấy 4-bit bên trái và R lấy 4-bit bên phải.
  - Gọi đến hàm thực thi 1 vòng của cấu trúc feistel
  - In ra kết quả sau khi thực hiện hàm feistel_round
  - Gộp nửa trái và nửa phải lại thành kết quả mã hóa cuối cùng bằng phép dịch bit trái (<<) và phép toán OR (|).
  - Chạy qua vòng lặp tiếp theo cho đến hết 4 vòng lặp.

Nhận xét:

Kết quả thấy sự khác biệt giữa hai bản rõ ban đầu chỉ 1-bit đã tăng dần qua từng vòng mã hóa. Sau mỗi vòng, thay đổi nhiều bit hơn và ảnh hưởng đến cả hai nửa của khối dữ liệu. Đến các vòng cuối, sự khác biệt gần như toàn bộ các bit đều bị thay đổi. Điều này

chứng minh thuật toán Feistel thể hiện rõ hiệu ứng thác đổ (Avalanche Effect), một đặc tính quan trọng đảm bảo tính an toàn của hệ mật mã.

2. Mode of Operation - Chế độ hoạt động

1. Ý tưởng:
   Sử dụng thư viện pycryptodome, viết script mã hóa chuỗi văn bản có tính lặp lại cao: "UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_". Thực hiện mã hóa bằng cả hai chế độ AES-ECB và AES-CBC (với IV cố định).
   - In kết quả dưới dạng Hexadecimal và so sánh các khối 16-byte của bản mã.
   - tìm hiểu, thử nghiệm tất cả các Mode of Operation khác, và đưa ra nhận xét.
2. Để giải quyết yêu cầu của bài toán
   - Tạo đối tượng mã hóa và thực hiện mã hóa ở chế độ AES-ECB.
   - Tạo đối tượng mã hóa và thực hiện mã hóa ở chê độ AES-CBC.
3. Ngôn ngữ sử dụng: Python 3.14
4. Các bước thực hiện:

Bước 1: import thêm AES dùng để tạo bộ mã hóa AES từ thư viện pycryptodome.
Bước 2: khai báo key là khóa AES 16 byte.
Khai báo thêm plaintext là chuỗi cần được mã hóa, chuỗi này lặp lại nhiều lần để thấy được sự khác biệt giữa ECB và CBC.

```
from Crypto.Cipher import AES

key = b'1234567890123456'
plaintext = b"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_"
```

Hình 2.1: import thêm AES từ thư viện pycryptodome và tạo key, plaintext.

Bước 3: Tạo đối tượng và thực hiện mã hóa ở chế độ AES-ECB.
Ý tưởng:
- Tạo object mã hóa AES với hàm new bên trong AES.
- ‘AES.MODE_ECB’ là chế độ electronic Codebook.
- ECB mã hóa ở mỗi block độc lập.
- Tiếp theo, thực hiện việc mã hóa đối tượng vừa được tọa ra với hàm encrypt có tham số đầu vào là plaintext cần được mã hóa.

```
# ECB
cipher_ecb = AES.new(key, AES.MODE_ECB)
ct_ecb = cipher_ecb.encrypt(plaintext)
```

Hình 2.2: thực hiện tạo đối tượng và mã hóa ở chế độ AES-ECB.

Bước 4: Tạo đối tượng và thực hiện mã hóa ở chế độ AES-CBC.

Ý tưởng:

- Khai báo iv (Initialization Vector) 16 bytes, ở chế độ CBC thì cần iv để bắt đầu chuỗi. Mỗi khối bản rõ sẽ XOR với khối bản mã trước đó rồi mới đem đi mã hóa. Do khối đầu tiên chưa có bản mã nào trước đó, nên cần lấy IV này để XOR.
- Tạo object mã hóa AES với hàm new bên trong AES.
- 'AES.MODE_CBC' là chế độ Cipher Block Chaining.
- Tiếp theo, thực hiện việc mã hóa đối tượng vừa được tọa ra với hàm encrypt có tham số đầu vào là plaintext cần được mã hóa.

```
# CBC
iv = b'0000000000000000'
cipher_cbc = AES.new(key, AES.MODE_CBC, iv)
ct_cbc = cipher_cbc.encrypt(plaintext)
```

Hình 2.3: thực hiện tạo đối tượng và mã hóa ở chế độ AES-CBC.

Bước 5: chương trình hoàn chỉnh và in ra chuỗi mã hóa ở chế độ AES-ECB và AES-CBC

```
Lab02 > Task2_2.py > ...
from Crypto.Cipher import AES

key = b'1234567890123456'
plaintext = b"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_"

# ECB
cipher_ecb = AES.new(key, AES.MODE_ECB)
ct_ecb = cipher_ecb.encrypt(plaintext)

# CBC
iv = b'0000000000000000'
cipher_cbc = AES.new(key, AES.MODE_CBC, iv)
ct_cbc = cipher_cbc.encrypt(plaintext)

print("ECB:", ct_ecb.hex())
print("CBC:", ct_cbc.hex())
```

Hình 2.4: Chương trình hoàn chỉnh để mã hóa bằng cả hai chế độ AES-ECB và AES-CBC.

Giải thích: hàm hex () dùng để in ra chuỗi mã hóa ở dạng thập lục phân.

1. In kết quả dưới dạng Hexadecimal.

```
PS D:\antoanmangmaytinh> & C:/Users/ADMIN/AppData/Local/Programs/Python/
sk2_2.py
ECB: 02b0647de210da452e3bdcadc415814e02b0647de210da452e3bdcadc415814e
CBC: 47747530a46ebd156c1da8763c54be1b46807a7ec513e5e69d9256881a0f0c09
```

Hình 2.5: In ra kết quả mã hóa ở 2 chế độ ECB và CBC.

2. So sánh các khối 16-byte của bản mã.

```
ECB blocks:
02b0647de210da452e3bdcadc415814e
02b0647de210da452e3bdcadc415814e

CBC blocks:
ecf142f95b525736d2d12247f3c647e5
58e5c9b9758b77eeaac1c100c30bdcfc
```

Hình 2.6 In ra kết quả các khối 16-byte ở 2 chế độ ECB và CBC.

3. Nhận xét:

- ECB: các khối plaintext giống nhau (UIT_LAB_UIT_LAB_), chế độ ECB mã hóa độc lập bằng cùng một khóa. Kết quả là 2 khối bản mã giống nhau, để lộ cấu trúc lặp của dữ liệu.
- CBC: Khối plaintext tiếp theo được XOR với khối ciphertext trước đó trước khi mã hóa. Kết quả là 2 khối ciphertext khác nhau dù plaintext lặp lại. Tính bảo mật cao hơn ECB.

3. Avalanche Effect

Task 2.3:

1. Ý tưởng: Viết script mã hóa hai chuỗi "STAYHOME" và "STAYHOMA" bằng thuật toán DES. Chuyển kết quả bản mã sang dạng chuỗi nhị phân và đếm số lượng bit khác nhau (Hamming Distance). Thay đổi key lần lượt với mã số sinh viên trong nhóm, tính toán tỷ lệ phần trăm thay đổi bit với các lần thay đổi và đưa ra nhận xét
2. Để giải quyết yêu cầu bài toán:

- Xây dựng hàm chuyển bytes sang bit.
- Xây dựng hàm tính Hamming distance để đếm số bit khác nhau giữa 2 chuỗi bit.
- Xây dựng hàm kiểm tra avalanche là hàm thực hiện toàn bộ yêu cầu của đề bài.

3. Ngôn ngữ sử dụng: Python 3.14
4. Các bước thực hiện:

Bước 1: thực hiện import thuật toán DES.

- DES có block size: 64 bits.
- Key: 8 bytes.

- Dùng để minh họa Alanvache Efect.

Bước 2: Xây dựng hàm chuyển bytes sang bit.

Ý tưởng:

- 'int.from_bytes(data, 'big')' để chuyển bytes sang số nguyên.
- bin () chuyển sang nhị phân.
- Zfill(64) để thêm số phía trước cho đủ 64 bit.

```
Lab02 > Task2_3.py > avalanche_test
from Crypto.Cipher import DES

def to_bits(data):
    return bin(int.from_bytes(data, 'big'))[2:].zfill(64)
```

Hình 3.1: import thuật toán DES và xây dựng hàm chuyển bytes sang bit.

Bước 3: Xây dựng hàm tính Hamming distance.

Ý tưởng: đầu vào là 2 chuỗi bit và đếm số bit khác nhau giữa 2 chuỗi bit.

- zip(a,b) ghép từng bit.
- Nếu 2-bit khác nhau thì hàm sum được thực thi và đếm số bit khác nhau.

```
def hamming_distance(a, b):
    return sum(x != y for x, y in zip(a, b))
```

Hình 3.2: hàm tính Hamming distance để đếm số bit khác nhau giữa 2 chuỗi bit.

Bước 4: Xây dựng hàm kiểm tra avalanche.

Ý tưởng: Thực hiện toàn bộ yêu cầu của đề bài.

- Khai báo key có độ dài 8 bytes dùng cố định để so sánh.
- Khai báo 2 plaintext khác nhau 1 kí tự.
- Tạo object để thực hiễn mã hóa DES với hàm new.
- 'DES.MODE_ECB' dùng ở chế độ ECB.
- Thực hiện mã hóa 2 plaintext ban đầu.
- Chuyển Ciphertext sang bit.
- Tính số bit khác nhau giữa 2 chuỗi.
- Tính phần trăm và in ra kết quả.
- Thực hiện gọi hàm avalanchetest() để thực hiện logic bên trong hàm.

```python
def avalanche_test():
    key = b'87654321'
    p1 = b'STAYHOME'
    p2 = b'STAYHOMA'

    cipher = DES.new(key, DES.MODE_ECB)

    c1 = cipher.encrypt(p1)
    c2 = cipher.encrypt(p2)

    b1 = to_bits(c1)
    b2 = to_bits(c2)

    diff = hamming_distance(b1, b2)
    percent = diff / 64 * 100

    print("Bit khác nhau:", diff)
    print("Tỷ lệ:", percent, "%")

avalanche_test()
```

Hình 3.3: code bên trong hàm avalanchetest.

Bước 5: Thực hiện chạy code đưa ra kết quả và nhận xét.

```
PS D:\antoanmangmaytinh> & C:/Use
sk2_3.py
Bit khác nhau: 37
Tỷ lệ: 57.8125 %
```

Hình 3.4: kết quả sau khi thực hiện chạy code.

Thực hiện đổi key thành mã số sinh viên '24520115':

```
sk2_3.pytoanmangmaytinh>
Bit khác nhau: 35
Tỷ lệ: 54.6875 %
```

Hình 3.5: kết quả khi thực hiện đổi key thành mã số sinh viên.

Nhận xét: chạy đoạn code trên, tỷ lệ thay đổi bit thường xấp xỉ 50% và trên 50%.

Task 2.4

1. Ý tưởng:
   SEED Lab - Secret-Key Encryption Lab:
   Thực hiện các bước sau bằng Python để hiểu rõ về đặc tính lan truyền lỗi (error propagation) của các chế độ mã hóa khác nhau:
   1. Tạo dữ liệu 1000 byte.
   2. Mã hóa dữ liệu bằng AES-128.
   3. Làm hỏng bản mã bằng cách đảo 1-bit tại byte thứ 26.
   4. Giải mã bản mã đã bị lỗi và quan sát có bao nhiêu khối dữ liệu bản rõ bị hỏng với chế độ mã hóa được sử dụng lần lượt là ECB, CBC, CFB, hoặc OFB.
2. Để thực hiện yêu cầu của bài toán:
   - Tạo dữ liệu 1000 byte.
   - Xây dựng hàm mã hóa dữ liệu bằng AES-128.
   - Xây dựng hàm làm hỏng bản mã bằng cách đảo 1-bit tại byte thứ 26.
   - Xây dựng hàm giải mã.
   - Xây dựng hàm đếm số block bị lỗi.
3. Ngôn ngữ sử dụng: Python 3.14
4. Các bước thực hiện:

Bước 1: thực hiện import AES để có thể sử dụng thuật toán AES.

- import get_random_bytes để có thể tọa key và iv ngẫu nhiên.
- import pad để thêm padding để dữ liệu chia hết cho block 16 bytes.
- 'BLOCK = 16' có nghĩa AES block size = 16-byte dùng để chia dữ liệu tành các block.

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

BLOCK = 16
```

Hình 3.6: import các thư viện và thuật toán cần thiết cho việc mã hóa và giải mã.

Bước 2: Tạo dữ liệu 1000 byte

- 'plaintext = b'A' * 1000' tạo dữ liệu 1000 bytes kí tự 'A'.
- 'data = pad (data, BLOCK)' padding dữ liệu để độ dài chia hết cho 16, BLOCK = 16.
- 'key = get_random_bytes (16)' tạo key AES-128 có 16 bytes
- 'iv = get_random_bytes (16)' tọa iv ngẫu nhiên dùng cho các chế độ như CBC, CFB, OFB.

```
# 1. tạo dữ liệu 1000 byte
data = b'A' * 1000
data = pad(data, BLOCK)

key = get_random_bytes(16)
iv = get_random_bytes(16)
```

Hình 3.7: Tạo các dữ liệu cần thiết cho việc mã hóa và giải mã.

Bước 3: Xây dựng hàm mã hóa dữ liệu bằng AES-128.

Ý tưởng:

- Chọn chế độ tương ứng
- Tọa ra object để mã hóa với hàm new với từng mode tương ứng.
- Mã hóa data với hàm encrypt.

```
# 2. mã hóa
if mode == "ECB":
    cipher = AES.new(key, AES.MODE_ECB)
elif mode == "CBC":
    cipher = AES.new(key, AES.MODE_CBC, iv)
elif mode == "CFB":
    cipher = AES.new(key, AES.MODE_CFB, iv)
elif mode == "OFB":
    cipher = AES.new(key, AES.MODE_OFB, iv)

ciphertext = cipher.encrypt(data)
```

Hình 3.8: tạo ra đối tượng tương ứng cho việc thực hiện việc mã hóa.

Bước 4: Xây dựng hàm làm hỏng bản mã bằng cách đảo 1-bit tại byte thứ 26.

Ý tưởng: đầu vào là data đã được padding và vị trí của bytes, vị trí của bit.

- Chuyển data đầu vào thành bytearray để sửa
- byte_index là vị trí byte cần sửa.
- bit_index là vị trí bit trong byte.
- '^=' XOR để đỏa bit.
- '1<< bit_index' tọa mask để lật bit.
- Trả về vị trí bytes cần sửa

```python
# Hàm đảo bit tại vị trí byte_index và bit_index
def flip_bit(data, byte_index, bit_index):
    corrupted = bytearray(data)
    corrupted[byte_index] ^= (1 << bit_index)
    return bytes(corrupted)
```

Hình 3.9: code bên trong hàm đảo bit tại byte thứ 26.

```python
    # 3. đảo 1 bit tại byte thứ 26
    corrupted = flip_bit(ciphertext, 25, 0)
```

Hình 3.10: gọi hàm đảo 1-bit để thực thi code có đầu vào là ciphertext, byte thứ 26 và bit thứ 0 và chỉ lật 1 bit.

Bước 5: Xây dựng hàm giải mã.

Ý tưởng: giải mã dữ liệu đã bị lỗi Ciphertext đã bị lật bit.

```python
    # 4. giải mã
    if mode == "ECB":
        cipher2 = AES.new(key, AES.MODE_ECB)
    elif mode == "CBC":
        cipher2 = AES.new(key, AES.MODE_CBC, iv)
    elif mode == "CFB":
        cipher2 = AES.new(key, AES.MODE_CFB, iv)
    elif mode == "OFB":
        cipher2 = AES.new(key, AES.MODE_OFB, iv)

    decrypted = cipher2.decrypt(corrupted)
```

Hình 3.11: thực hiện việc giải mã với Ciphertext đã bị lỗi.

Bước 6: Xây dựng hàm đếm số block bị lỗi.

Ý tưởng: đầu vào bao gồm plaintext gốc và plaintext bị lỗi.

- ‘blocks = len(p1) // BLOCK’ tính tổng số block.
- ‘for i in range(blocks):’ duyệt qua từng block.
- p1[i*BLOCK:(i+1)*BLOCK] cắt block thứ I ra để so sánh.
- nếu 2 block khác nhau thì là block lỗi.
- trả về kết quả số block lỗi.

```python
# Hàm đếm số block bị hỏng
def count_corrupted_blocks(p1, p2):
    blocks = len(p1) // BLOCK
    count = 0
    for i in range(blocks):
        if p1[i*BLOCK:(i+1)*BLOCK] != p2[i*BLOCK:(i+1)*BLOCK]:
            count += 1
    return count
```

Hình 3.12: Hàm đếm số block lỗi.

```python
    # đếm block bị lỗi
    corrupted_blocks = count_corrupted_blocks(data, decrypted)

    print("Số block bị lỗi:", corrupted_blocks)

for m in ["ECB", "CBC", "CFB", "OFB"]:
    test_mode(m)
```

Hình 3.13: gọi hàm đếm số block lỗi và in ra số block lỗi với từng chế độ.

Bước 7: thực hiện chạy code và xem kết quả sau đó đưa ra nhận xét.

```
PS D:\antoanmangmaytinh> & C

Mode: ECB
Số block bị lỗi: 1

Mode: CBC
Số block bị lỗi: 2

Mode: CFB
Số block bị lỗi: 2

Mode: OFB
Số block bị lỗi: 1
PS D:\antoanmangmaytinh>
```

Hình 3.14: Kết quả sau khi chạy code in ra số block bị lỗi của từng mode.

Nhận xét:

- ECB: Hỏng 1 khối. Khối chứa lỗi bị hỏng ngẫu nhiên hoàn toàn. Lỗi không lây sang khối sau.

- CBC: Hỏng 2 khối. Khối chứa lỗi hỏng hoàn toàn, khối ngay sau nó bị lật đúng 1-bit tương ứng.
- CFB: Hỏng 2 khối. Khối chứa lỗi bị lật đúng 1 bit, nhưng khối ngay sau nó lại hỏng hoàn toàn.
- OFB: Lỗi 1 bit. Sai đúng 1-bit ở vị trí tương ứng trong khối, không lây lan sang phần khác.

Task 2.5

Tìm hiểu và so sánh độ mạnh và sự khác biệt của DES, Triple-DES (3DES), và AES. Tại sao chúng ta không nên sử dụng mã hóa double-DES (2DES)?

1. So sánh

Đặc điểm	DES (Data Encryption Standard)	3DES (Triple-DES)	AES (Advanced Encryption Standard)
Năm	1977	1998	2001
Kích thước	64-bit	64-bit	128-bit
Độ dài khóa	56-bit	112-bit hoặc 168-bit	128, 192, hoặc 256-bit
Tốc độ xử lý	Nhanh (đã lỗi thời)	Rất chậm (chạy DES 3 lần)	Rất nhanh (hỗ trợ tăng tốc phần cứng)
Cấu trúc thuật toán	Feistel Network	Feistel Network	Substitution-Permutation Network (SPN)
Mức độ an toàn	Không an toàn (có thể bị phá bởi Brute-force)	Đã lỗi thời (Dễ bị tấn công Sweet32)	Rất an toàn (Tiêu chuẩn toàn cầu hiện nay)

2. Sự khác biệt của DES, Triple-DES (3DES), và AES
   - DES: Vấn đề lớn nhất của DES là không gian khóa quá nhỏ (chỉ có 2^56 trường hợp). Với sức mạnh máy tính hiện nay, DES có thể bị phá bằng phương pháp vét cạn (brute-force) chỉ trong vài giờ.

- 3DES: Để khắc phục điểm yếu của DES mà không phải thiết kế lại toàn bộ hệ thống phần cứng/phần mềm cũ, người ta tạo ra 3DES bằng cách chạy thuật toán DES 3 lần liên tiếp: Mã hóa (K1) -> Giải mã (K2) -> Mã hóa (K3). Giúp tăng độ dài khóa lên 168-bit. Tuy nhiên, 3DES rất tốn tài nguyên, chạy chậm chạp và kích thước khối 64-bit của nó khiến nó dễ bị tấn công va chạm (collision attacks) nếu mã hóa lượng dữ liệu lớn.
- AES: Được thiết kế lại hoàn toàn từ đầu để thay thế DES/3DES. AES sử dụng kích thước khối lớn hơn (128-bit), khóa dài hơn và cấu trúc toán học tối ưu hơn, giúp vừa an toàn tuyệt đối trước các cuộc tấn công vét cạn hiện tại, vừa cực kỳ nhẹ và nhanh gọn.

3. Tại sao chúng ta không nên sử dụng mã hóa double-DES (2DES)?

a. Vấn đề:

- Thay vì chạy 3 lần (3DES) quá chậm, tại sao không mã hóa 2 lần (2DES) với 2 khóa khác nhau (K1 và K2)?

Công thức sẽ là: $C = E_{k2}(E_{k1}(P))$

b. Kỳ vọng:

- Người ta từng kỳ vọng 2 khóa 56-bit kết hợp lại sẽ tạo ra độ an toàn là 2^112.

c. Lổ hỏng:

- Tuy nhiên, 2DES đã bị đánh bại hoàn toàn bởi một kỹ thuật gọi là Tấn công gặp nhau ở giữa (Meet-in-the-Middle Attack). Cách cuộc tấn công này hoạt động:
- Kẻ tấn công chỉ cần có trong tay một cặp Bản rõ (Plaintext - P) và Bản mã (Ciphertext - C) tương ứng.
    - Từ phía Bản rõ (P): Kẻ tấn công dùng tất cả 2^56 trường hợp của khóa K1 để mã hóa P, tạo ra một bảng chứa 2^56 giá trị trung gian.
    - Từ phía Bản mã (C): Kẻ tấn công dùng tất cả 2^56 trường hợp của khóa K2 để giải mã C, tạo ra các giá trị trung gian khác.
    - Gặp nhau ở giữa: Kẻ tấn công so sánh hai tập hợp kết quả trên. Nếu có một giá trị khớp nhau, đó chính là cặp khóa (K1, K2) cần tìm.

d. Hậu quả:

- Thay vì phải thử 2^112 trường hợp, kẻ tấn công chỉ cần thực hiện khoảng 2^56 + 2^56 = 2^57 phép tính.
- Có nghĩa là, độ an toàn của 2DES thực chất chỉ nhỉnh hơn DES chuẩn đúng 1 bit (2^57 so với 2^56). Đó là lý do thế giới phải bỏ qua 2DES và tiến thẳng lên 3DES (vì 3DES miễn nhiễm với kiểu tấn công này).

4. Mersenne Prime - Số nguyên tố lớn

1. Ý tưởng:
   Viết một chương trình (sử dụng ngôn ngữ lập trình) để đáp ứng các yêu cầu sau:
   1. Số nguyên tố
      i. Tọa ra số nguyên tố ngẫu nhiên có 8 bits, 16 bits, 64 bits.
      ii. Xác định 10 số nguyên tố lớn nhất nhỏ hơn số nguyên tố Mersenne thứ 10.
      iii. Kiểm tra xem một số nguyên tùy ý nhỏ hơn 2^89 – 1 có phải là số nguyên tố hay không?
   2. Xác định ước số chung lớn nhất (GCD) của 2 số nguyên lớn tùy ý (lớn nhất có thể xử lý).
   3. Tính toán lũy thừa theo module a^x mod p. Chương trình phải có khả năng tính toán trong trường hợp số mũ lớn (x>40), ví dụ, 7^40 mod 19.

Gợi ý:

- Để kiểm tra các số nguyên tố lớn (Mersenne Prime), sử dụng thuật toán Miller-Rabin.
- Để xác định ước số chung lớn nhất (greatest common divisor), sử dụng thuật toán Euclid.
- Phép nhân module có tính chất sau:

$$(a \times b) \equiv [(a \bmod n) \times (b \bmod n)] \bmod n$$

Tính chất này có thể được áp dụng để tính toán lũy thừa module (modular exponentiation) trong trường hợp có số mũ lớn.

2. Để thực hiện yêu cầu của bài toán:
   - Xây dựng hàm Miller-Rabin (kiểm tra nguyên tố).
   - Xây dựng hàm Sinh số nguyên tố ngẫu nhiên.
   - Xây dựng hàm Tìm 10 số nguyên tố nhỏ hơn Mersenne thứ 10.
   - Xây dựng hàm tìm ước chung lớn nhất GCD (Euclid).
   - Xây dựng hàm Lũy thừa modulo.
3. Ngôn ngữ sử dụng: Python
4. Các bước thực hiện:

Bước 1: Xây dựng hàm Miller-Rabin (kiểm tra nguyên tố).

Ý tưởng: Hàm miller_rabin (n, k=5) dùng để kiểm tra một số nguyên n có phải là số nguyên tố theo phương pháp xác suất.

- Đầu tiên loại bỏ các trường hợp đơn giản:
  - Nếu n < 2 thì không phải số nguyên tố.
  - Nếu n = 2 hoặc 3 thì là số nguyên tố.
  - Nếu n chẵn thì không phải số nguyên tố.

```
if n < 2:
    return False
if n in (2, 3):
    return True
if n % 2 == 0:
    return False
```

Hình 4.1: Loại bỏ 1 số trường hợp đơn giản đối với số nguyên tố.

- Tiếp theo, thuật toán phân tích n – 1 thành dạng: n – 1 = 2^s * d. với mục đích đưa bài toán kiểm tra nguyên tố về dạng kiểm tra lũy thừa modulo.
    - d là số lẻ, s là số lần n – 1 chia cho 2.

```
# n-1 = 2^s * d
d = n - 1
s = 0
while d % 2 == 0:
    d //= 2
    s += 1
```

Hình 4.2: phân tích n – 1 thành dạng n – 1 = 2^s * d.

- Tiếp theo lặp k lần để kiểm tra:
    - Chọn ngẫu nhiên một số a trong khoảng [2, n – 2]
    - Tính x = a^d mod n
    - Nếu x = 1 hoặc x = n – 1 thì n có khả năng là số nguyên tố
    - Nếu không, tiếp tục bình phương x theo modulo n tối đa s – 1 lần
    - Nếu x trở thành n – 1 thì tiếp tục vòng lặp
    - Nếu không xảy ra điều kiện trên thì kết luận n là hợp số
    - Sau k lần kiểm tra mà không phát hiện hợp số thì n được coi là số nguyên tố với xác suất cao.

```python
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False

    return True
```

Hình 4.3: lặp lại k lần để kiểm tra xem n có phải là số nguyên tố với xác suất cao.

```python
# 1. Miller-Rabin (kiểm tra nguyên tố)
def miller_rabin(n, k=5):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False

    # n-1 = 2^s * d
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1

    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)

        if x == 1 or x == n - 1:
            continue

        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False

    return True
```

Hình 4.4: Toàn bộ phần code logic bên trong hàm kiểm tra số nguyên tố.

Bước 2: Xây dựng hàm Sinh số nguyên tố ngẫu nhiên.

Ý tưởng: Hàm generate_prime(bits) dùng để sinh số nguyên tố có độ dài bits.

- Chương trình thực hiện các các bước sau:
- Sinh ngẫu nhiên số có bits bằng hàm random.getrandbits(bits).
- Đặt bit cao nhất bằng 1 để đảm bảo độ dài đúng.
- Đặt bit thấp nhất bằng 1 để đảm bảo số lẻ.
- Kiểm tra số vừa sinh bằng Miller–Rabin.
- Nếu là số nguyên tố thì trả về, nếu không thì lặp lại.

```
# 2. Sinh số nguyên tố ngẫu nhiên
def generate_prime(bits):
    while True:
        num = random.getrandbits(bits)
        num |= (1 << bits - 1) | 1  # đ
        if miller_rabin(num):
            return num
```

Hình 4.5: Code logic bên trong hàm sinh ngẫu nhiên số nguyên tố.

Bước 3: Nếu là số nguyên tố thì trả về, nếu không thì lặp lại

Ý tưởng: Hàm find_primes_below_mersenne() tìm 10 số nguyên tố nhỏ hơn số 2^89-1.

- Chương trình bắt đầu từ giá trị nhỏ hơn số Mersenne và giảm dần:
  - Nếu số hiện tại là nguyên tố thì đưa vào danh sách.
  - Tiếp tục cho đến khi tìm đủ 10 số nguyên tố.
  - Trả về danh sách bao gồm 10 số nguyên tố vừa tìm ra.

```
# 3. Tìm 10 số nguyên tố nhỏ hơn Mersenne thứ 10
def find_primes_below_mersenne(count=10):
    mersenne_10 = 2**89 - 1
    primes = []
    num = mersenne_10 - 1

    while len(primes) < count:
        if miller_rabin(num):
            primes.append(num)
        num -= 1

    return primes
```

Hình 4.6: Code bên trong hàm tìm 10 số nguyên tố nhỏ hơn Mersenne thứ 10.

Bước 4: Xây dựng hàm tìm ước chung lớn nhất GCD (Euclid).
Ý tưởng: Hàm gcd(a, b) tính ước chung lớn nhất của hai số nguyên bằng thuật toán Euclid.

- Thuật toán dự trên tính chất: gcd(a,b) = gcd(b, a mod b)
  - Chương trình lặp qua cho đến khi b = 0.
  - Thay a = b.
  - Thay b = a mod b.
  - Khi b = 0 thì a chính là ước chung lớn nhất.

```
# 4. GCD (Euclid)
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

Hình 4.7: Code bên trong hàm gcd để tìm ước chung lớn nhất của 2 số.

Bước 5: Xây dựng hàm Lũy thừa modulo.
Ý tưởng: Hàm mod_exp(a, x, p) có nhiệm vụ tính a^x mod p.

- Thuật toán thực hiện các bước:
  - Nếu x lẻ thì nhân kết quả với a.
  - Bình phương a.
  - Chia đôi x.
  - Lặp lại cho đến chi x = 0.

```
# 5. Lũy thừa modulo
def mod_exp(a, x, p):
    result = 1
    a = a % p

    while x > 0:
        if x % 2 == 1:
            result = (result * a) % p
        a = (a * a) % p
        x //= 2

    return result
```

Hình 4.8: Code bên trong hàm lũy thừa modulo.

Bước 6: Chương trình thực thi trong hàm main.

Ý tưởng: là hàm chính để chạy chương trình.

- Chương trình thực hiện:
    - Sinh số nguyên tố 8 bit, 16 bit và 64 bit.
    - Tìm 10 số nguyên tố nhỏ hơn số Mersenne.
    - Cho người dùng nhập số để kiểm tra nguyên tố.
    - Tính GCD của hai số nhập vào.
    - Tính lũy thừa modulo theo dữ liệu người dùng nhập.

Hình ảnh bên dưới là code logic bên trong hàm main:

```python
# MAIN
if __name__ == "__main__":
    print("1. Sinh số nguyên tố")
    prime8 = generate_prime(8)
    prime16 = generate_prime(16)
    prime64 = generate_prime(64)

    print("Prime 8-bit :", prime8)
    print("Prime 16-bit:", prime16)
    print("Prime 64-bit:", prime64)

    print("\n2. 10 số nguyên tố < 2^89 - 1")
    primes = find_primes_below_mersenne()
    for i, p in enumerate(primes, 1):
        print(f"{i}: {p}")

    print("\n3. Kiểm tra số nguyên tố")
    test_num = int(input("Nhập số cần kiểm tra: "))
    if miller_rabin(test_num):
        print("=> Là số nguyên tố")
    else:
        print("=> Không phải số nguyên tố")

    print("\n4. GCD")
    a = int(input("Nhập a: "))
    b = int(input("Nhập b: "))
    print("GCD =", gcd(a, b))

    print("\n5. Lũy thừa modulo")
    a = int(input("Nhập a: "))
    x = int(input("Nhập x: "))
    p = int(input("Nhập p: "))
    print(f"{a}^{x} mod {p} =", mod_exp(a, x, p))
```

Bước 7: chạy chương trình nhập dữ liệu và kiểm tra kết quả.

```
PS D:\antoanmangmaytinh\Lab02> python .\Task2_6.py
1. Sinh số nguyên tố
Prime 8-bit : 151
Prime 16-bit: 34183
Prime 64-bit: 11425338897908186219

2. 10 số nguyên tố < 2^89 - 1
1: 618970019642690137449562091
2: 618970019642690137449562081
3: 618970019642690137449562063
4: 618970019642690137449562043
5: 618970019642690137449562013
6: 618970019642690137449562009
7: 618970019642690137449561847
8: 618970019642690137449561791
9: 618970019642690137449561671
10: 618970019642690137449561509

3. Kiểm tra số nguyên tố
Nhập số cần kiểm tra: 97
=> Là số nguyên tố

4. GCD
Nhập a: 14
Nhập b: 28
GCD = 14

5. Lũy thừa modulo
Nhập a: 7
Nhập x: 40
Nhập p: 19
7^40 mod 19 = 7
```

Hình 4.10: Kết quả sau khi chạy code.

Giải thích:

- Sinh ra số nguyên tố có độ dài 8 bits là: 151
    - Chuyển sang hệ nhị phân: 1001 0111
- Sinh ra số nguyên tố có độ dài 16 bits là: 34183
    - Chuyển sang hệ nhị phân: 1000 0101 1000 0111
- Sinh ra số nguyên tố có độ dài 64 bits là: 11425338897908186219
- Sinh ra 10 số nguyên tố nhỏ hơn số Mersenne thứ 10:

```
2. 10 số nguyên tố < 2^89 - 1
1: 618970019642690137449562091
2: 618970019642690137449562081
3: 618970019642690137449562063
4: 618970019642690137449562043
5: 618970019642690137449562013
6: 618970019642690137449562009
7: 618970019642690137449561847
8: 618970019642690137449561791
9: 618970019642690137449561671
10: 618970019642690137449561509
```

- Kiểm tra 1 số được nhập vào có phải là số nguyên tố hay không:
    - Đầu vào là kiểm tra số 97
    - Kết quả là số 97 là số nguyên tố => đúng

```
3. Kiểm tra số nguyên tố
Nhập số cần kiểm tra: 97
=> Là số nguyên tố
```

- Tìm ước chung lớn nhất của 2 số nhập vào từ bàn phím:
    - Số thứ nhất là: 14
    - Số thứ hai là: 28
    - Ước chung của 2 số trên là: 1, 2, 7, 14
    - Kết quả ước chung lớn nhất của 2 số là: 14

```
4. GCD
Nhập a: 14
Nhập b: 28
GCD = 14
```

- Tìm lũy thừa modulo
    - Nhập a = 7

- Nhập x = 40
- Nhập p = 19
- Kết quả của a^x mod p là 7^40 mod 19 là 7

```
5. Lũy thừa modulo
Nhập a: 7
Nhập x: 40
Nhập p: 19
7^40 mod 19 = 7
```